

ALPHA LENS

Comprehensive Testing Plan

Pre-Production Quality Assurance & User Acceptance Testing

Version 1.0 · March 2026

Testing Team	Location	Focus Area	Size
Philippines Team	Manila / Remote	Functional, API, International access	TBD
Finance Students	Local University	UX, Paper trading, Recommendations	TBD
Core Team (You)	US	Live trading, Kalshi, Risk controls	1

1. Testing Overview

This plan covers pre-production testing of Alpha Lens — the AI-powered global investment intelligence platform. Testing is organized into four phases across three tester groups: the Philippines team (technical and functional testing), local finance students (user experience and learning module testing), and the core team (live trading and risk control validation).

Goal

Ensure Alpha Lens is stable, accurate, and intuitive before opening to real users and live trading capital. Every critical path — from market scanning through AI recommendations to paper and live trade execution — must be verified by at least two independent testers.

1.1 Testing Phases

Phase	Name	Duration	Who	What
1	Infrastructure & Setup	Week 1	Philippines team	Environment, APIs, database, all integrations connecting
2	Functional Testing	Weeks 2–3	Philippines team + Core	Every feature, every screen, every API endpoint
3	User Acceptance Testing	Week 4	Finance students	Real usage scenarios, paper trading
4	Pre-Launch Validation	Week 5	All teams	Live trading dry run, load test, security review, sign-off

1.2 Devices and Environments

Alpha Lens must work on all of these. Each tester should test on at least two:

Device	Browser	Priority	Assigned to
Desktop / Laptop	Chrome 120+	Critical	All teams
Desktop / Laptop	Firefox, Edge, Safari	High	Philippines team
Android smartphone	Chrome Mobile	Critical	Finance students + Philippines
iPhone / iPad	Safari Mobile	High	Finance students
Tablet (any)	Any browser	Medium	Finance students
Slow connection (3G sim)	Chrome	High	Philippines team — real-world condition

2. Tester Roles and Responsibilities

2.1 Philippines Team

The Philippines team handles technical and functional testing. They have access to Kalshi and Polymarket (Philippines is not on either platform's blocked list), making them ideal for testing the full live trading pipeline in a non-US jurisdiction.

Role	Responsibilities	Access Level
Lead QA Engineer	Coordinates test execution, tracks bug reports, signs off on each phase, manages test data	Full admin
API Tester	Tests all backend endpoints using Postman or curl, validates request/response schemas, stress tests	API + DB read
Frontend Tester	Tests all pages on multiple devices and browsers, documents UI bugs with screenshots	App user
Live Trading Tester	Executes paper and (small) live trades on Kalshi, validates platform router, tests approval flow	Kalshi account
Data Validator	Verifies AI probability scores, checks evidence records for quality, validates recommendation logic	DB read access

2.2 Local Finance Students

Finance students test from the perspective of real end users who are learning to trade. They should have no prior briefing on how the app works — their confusion points are your UX bugs. Each student gets a fresh \$10,000 paper trading account.

Group	Background	Testing Focus	Guidance
-------	------------	---------------	----------

Beginner group (no trading exp)	General finance students	Onboarding, paper trading intuition	No briefing — observe where they get stuck
Intermediate group (some trading)	Upper-year finance students	Scanner, AI Coach, recommendations, edge scoring	Brief intro to prediction markets only
Advanced group (trading exp)	Graduate students / finance majors	Full platform, historical data, signal quality critique	Full platform access, invite critique

Key Instruction for Finance Students:

Paper trading means NO real money changes hands. The \$10,000 balance is simulated. Clicking ‘Execute paper trade’ does not connect to any broker or exchange. It is entirely contained within the Alpha Lens app on their device.

2.3 Core Team (You)

- Validates Kalshi live trading integration with real account (small amounts — \$10 test trades)
- Tests US jurisdiction mode — Polymarket correctly shows paper-only, Kalshi routes correctly
- Validates all risk controls: daily loss limit, position cap, approval gate
- Reviews AI recommendation quality and coach commentary for accuracy
- Final sign-off authority for each phase

3. Phase 1 — Infrastructure & Setup Testing (Week 1)

Before any feature testing, verify that every external service is correctly connected and the app starts cleanly. The Philippines team runs this phase entirely.

3.1 Environment Checklist

Check	How to verify	Pass criterion	Tester
Replit app starts	Click Run, watch console	No errors, server listens on assigned PORT	PH Lead
Health endpoint	GET /api/healthz	Returns {"status": "ok"}	PH API
PostgreSQL connected	GET /api/markets — check response	Returns {"markets": [...], "total": N} from DB, no connection errors	PH API
Anthropic AI proxy valid	POST /api/markets/1/score	Returns {"market": {...}, "signals": [...], "scoring": {...}} , no auth error	PH API
Market data with sector filter	GET /api/markets?sector=crypto	Only crypto-sector assets returned	PH Data
Market sort options work	GET /api/markets?sort=price_change then ?sort=name then ?sort=alpha_score	List re-sorts correctly for each option	PH API

Seed data loaded	<code>GET /api/markets</code> after running seed script	12+ sample markets returned with prices and sectors	PH Lead
Frontend loads on all browsers	Visit app URL in Chrome, Firefox, Safari, Mobile	No blank screens, no console errors	PH Frontend
All nav pages load	<code>/ /market/1 /portfolio /coach /briefing /radar</code>	All 6 pages render without 404 or crash	PH Frontend
Seed data present	Run <code>cd artifacts/api-server && pnpm exec tsx src/seed.ts</code>	12 sample market assets and 11 evidence signals created	PH Lead
API codegen up to date	<code>pnpm --filter @workspace/api-spec run codegen</code>	Codegen completes with no errors, generated files match spec	PH Lead
TypeScript typecheck passes	<code>pnpm run typecheck</code>	Zero type errors across all packages	PH Lead

3.2 Scheduler Startup Verification

Check	How to verify	Pass criterion	Tester
Scheduler starts on boot	Watch server console after <code>Run</code>	Console shows <code>"Scheduler started: markets(5min) · recommendations(30min) · radar(5min)"</code>	PH Lead
Initial market refresh fires	Watch console ~3 seconds after boot	Console shows <code>"Running initial market data refresh..."</code> followed by price update logs	PH Lead
Initial refresh completes	Check for <code>"Market data refresh complete"</code> log	Log shows <code>{updated: N, total: N}</code> with <code>updated > 0</code>	PH Lead
No crash on initial refresh failure	Start app with no internet connection	Console logs error <code>"Scheduled market refresh failed"</code> , app stays running	PH API

3.3 External Data Source Verification

Check	How to verify	Pass criterion	Tester
CoinGecko API reachable	<code>GET /api/markets</code> — check crypto prices	BTC, ETH, SOL have non-null <code>currentPrice</code> with recent <code>updatedAt</code>	PH Data
Yahoo Finance API reachable	<code>GET /api/markets</code> — check stock/commodity prices	SPY, QQQ, GLD, USO, UNG, EURUSD have non-null <code>currentPrice</code>	PH Data
CoinGecko 24h change	Check crypto assets after market refresh	<code>priceChange24h</code> populated with realistic percentage (not null or 0)	PH Data
Yahoo price change calc	Check stock/commodity assets	<code>priceChange24h</code> = $((\text{current} - \text{previousClose}) / \text{previousClose}) * 100$, rounded to 2 decimal places	PH Data

10-second fetch timeout	Simulate slow network or block CoinGecko	Request aborts after 10s, logs warning, app continues without crash	PH API
-------------------------	--	---	--------

3.4 Kalshi Integration Check (Philippines team — live account)

Check	How to verify	Pass criterion
Trading accounts endpoint	GET /api/trading/accounts	Returns {"accounts": {"usJurisdictionMode": true/false, "primaryPlatform": "...", "kalshi": {...}, "alpaca": {...}, "polymarket": {...}}}
Kalshi status — configured	Set KALSHI_EMAIL and KALSHI_PASSWORD in Secrets, GET /api/trading/accounts	accounts.kalshi.status = "configured", legalStatus = "CFTC regulated — legal for US residents in all 50 states", depositMethod = "USD wire / bank transfer"
Kalshi status — not configured	Remove Kalshi Secrets, GET /api/trading/accounts	accounts.kalshi.status = "not_configured", message = "Add KALSHI_EMAIL and KALSHI_PASSWORD to Secrets", priority = "PRIMARY — set this up first"
Alpaca status — configured	Set ALPACA_API_KEY and ALPACA_SECRET_KEY, check response	accounts.alpaca.status = "configured", assetTypes = "US stocks and ETFs"
Alpaca status — not configured	Remove Alpaca Secrets, check response	accounts.alpaca.status = "not_configured", priority = "SECONDARY — for stock/ETF recommendations"
Polymarket — US mode	Set US_JURISDICTION_MODE=true, check response	accounts.polymarket.legalStatus = "PAPER TRADING ONLY (US jurisdiction mode ON)"
Polymarket — non-US mode	Set US_JURISDICTION_MODE=false, check response	accounts.polymarket.legalStatus = "Live trading enabled (non-US jurisdiction)" or "Available"
US jurisdiction mode flag	Check accounts.usJurisdictionMode	Matches the US_JURISDICTION_MODE env var (defaults to true)
Platform router selects correctly	GET /api/trading/route/{recommendation-id}	Returns {"selectedPlatform": "...", "reason": "...", "tradeable": true/false, "usJurisdictionMode": ..., "requireApproval": ...}

3.5 Radar Data Source Verification

Check	How to verify	Pass criterion	Tester
Radar status endpoint	GET /api/radar/status	Returns source list with 5 sources: coingecko, yahoo_finance, unusual_whales, alpha_vantage, finnhub	PH API

CoinGecko always active	Check <code>sources.coingecko.status</code>	Always <code>"active"</code> , tier <code>"free"</code>	PH API
Yahoo Finance always active	Check <code>sources.yahoo_finance.status</code>	Always <code>"active"</code> , tier <code>"free"</code>	PH API
Unusual Whales conditional	Check <code>sources.unusual_whales.status</code>	<code>"active"</code> if <code>UNUSUAL_WHALES_KEY</code> set, otherwise <code>"not_configured"</code>	PH API
Alpha Vantage conditional	Check <code>sources.alpha_vantage.status</code>	<code>"active"</code> if <code>ALPHA_VANTAGE_KEY</code> set, otherwise <code>"not_configured"</code>	PH API
Finnhub conditional	Check <code>sources.finnhub.status</code>	<code>"active"</code> if <code>FINNHUB_KEY</code> set, otherwise <code>"not_configured"</code>	PH API
Assets monitored count	Check <code>assetsMonitored</code>	Returns 18 (number of assets in <code>SPIKE_THRESHOLDS</code> map)	PH API
Chain maps count	Check <code>chainMaps</code>	Returns 8 (number of assets with chain reaction definitions)	PH API

Phase 1 Exit Criteria:

All environment checks pass with green status. Scheduler starts and fires initial refresh. External data sources return prices. Trading accounts endpoint returns valid platform statuses based on configured Secrets. Radar status shows at least 2 active sources (CoinGecko + Yahoo). App loads on at least 3 different browsers with zero console errors. TypeScript typecheck passes with zero errors. Only then move to Phase 2.

4. Phase 2 — Functional Testing (Weeks 2-3)

Systematic testing of every feature. Each test case has a unique ID, steps, expected result, and a pass/fail column for the tester to fill in. All bugs get logged in a shared tracking sheet.

4.1 Market Scanner (Homepage /)

Endpoint: `GET /api/markets`

Query params: `sector` (enum: energy, metals, agriculture, crypto, equities, fx, real_estate, prediction), `sort` (enum: alpha_score, price_change, name — default: alpha_score), `limit` (number — default: 50)

ID	Test case	Steps	Expected result	P/F
SC-01	Markets load on open	Navigate to <code>/</code> (homepage)	Markets list appears within 3 seconds, shows <code>markets</code> array and <code>total</code> count	
SC-02	Sector filter — energy	<code>GET /api/markets?sector=energy</code>	Only energy-sector assets returned	
SC-03	Sector filter — crypto	<code>GET /api/markets?sector=crypto</code>	Only crypto-sector assets returned	

SC-04	Sector filter — prediction	GET /api/markets?sector=prediction	Only prediction market assets returned	
SC-05	Sort by alpha score (default)	GET /api/markets or ?sort=alpha_score	Markets ordered by alphaScore descending	
SC-06	Sort by price change	GET /api/markets?sort=price_change	Markets ordered by priceChange24h descending	
SC-07	Sort by name	GET /api/markets?sort=name	Markets ordered alphabetically ascending	
SC-08	Limit parameter	GET /api/markets?limit=5	Exactly 5 markets returned (or fewer if less exist)	
SC-09	Edge badge colors	Find markets with positive and negative edge	Green badge for + edge, red for negative, gray for near-zero	
SC-10	Detail navigation	Click any market row	Asset detail page loads for correct market	
SC-11	Market fields complete	Check any market object in response	Contains: id , name , symbol , sector , currentPrice , priceChange24h , alphaScore , aiProbability , marketProbability , edge , direction , lastScoredAt , aiSummary , tradingBloc , riskLevel , updatedAt	
SC-12	Direction values	Check market direction field	Only returns "bullish" , "bearish" , "neutral" , or null	
SC-13	Risk level values	Check market riskLevel field	Only returns "low" , "medium" , "high" , "extreme" , or null	
SC-14	Mobile layout	Open scanner on smartphone	No horizontal scroll, text readable, buttons tappable	

4.2 Asset Detail & AI Scoring

Endpoints: GET /api/markets/:id , POST /api/markets/:id/score , POST /api/markets/refresh

ID	Test case	Steps	Expected result	P/F
AD-01	Market detail loads	GET /api/markets/1	Returns {"market": {...}, "signals": [...], "relatedMarkets": [...]}	
AD-02	Related markets shown	Check relatedMarkets in detail response	Returns markets from the same sector , excluding the current market	
AD-03	Signals attached	Check signals array in detail response	Returns up to 20 signals ordered by createdAt descending	

AD-04	Signal fields complete	Check any signal object	Contains: id , assetId , type , source , headline , detail , impact , direction , confidence , createdAt	
AD-05	Signal type values	Check signal type field	Only: "geopolitical" , "economic" , "technical" , "sentiment" , "fundamental" , "alternative"	
AD-06	Signal impact values	Check signal impact field	Only: "high" , "medium" , "low"	
AD-07	404 on invalid market	GET /api/markets/99999	Returns {"error": "Market not found"} with 404 status	
AD-08	AI scoring works	POST /api/markets/1/score	Returns {"market": {...}, "signals": [...], "scoring": {"aiProbability": N, "marketProbability": N, "edge": N, "direction": "...", "confidence": N, "reasoning": "..."} }	
AD-09	Scoring updates market	After scoring, GET /api/markets/1	Market alphaScore , aiProbability , edge , direction , riskLevel , aiSummary , lastScoredAt all updated	
AD-10	Scoring 404 on invalid	POST /api/markets/99999/score	Returns {"error": "Market not found"} with 404 status	
AD-11	Refresh all markets	POST /api/markets/refresh	Returns {"refreshed": N, "message": "Refreshed N markets with live data"}	
AD-12	Evidence direction colors	Check evidence cards on frontend	Green = bullish, red = bearish, gray = neutral	
AD-13	Probability gauge renders	Open any scored market on frontend	Gauge needle at correct position, color matches direction	
AD-14	Scoring generates signals	Score a market, then GET /api/signals/{assetId}	3–6 new signals created by AI with valid type , source , headline , impact , direction , confidence	
AD-15	Edge calculation	Score a market, check math	edge = aiProbability – marketProbability (rounded to 1 decimal)	
AD-16	Alpha score formula	Check scored market alphaScore	alphaScore =	edge
AD-17	Risk level thresholds	Score multiple markets, check riskLevel	"extreme" if	edge
AD-18	AI scoring fallback	Block Anthropic proxy (temporarily), POST	Returns a result (not 500): confidence: 0.3 , fallback probability $\approx 50 \pm 10$, reasoning	

		/api/markets/1/score	starts with "Fallback scoring applied"	
AD-19	Fallback direction logic	Check fallback result direction	"bullish" if fallback edge > 0, "bearish" if < 0, "neutral" if = 0	

4.3 Market Data Refresh Service

Tested via POST /api/markets/refresh and scheduler (every 5 minutes)

ID	Test case	Steps	Expected result	P/F
MD-01	CoinGecko crypto mapping	Refresh, check BTC/ETH/SOL prices	BTC CoinGecko bitcoin , ETH ethereum , SOL solana — all have fresh prices	
MD-02	Yahoo Finance stock mapping	Refresh, check SPY/QQQ/GLD/USO/UNG/EURUSD prices	SPY Yahoo SPY , QQQ QQQ , GLD GLD , USO USO , UNG UNG , EURUSD EURUSD=X — all have fresh prices	
MD-03	24h change — crypto	Check priceChange24h on a crypto asset	Value from CoinGecko usd_24h_change , rounded to 2 decimal places	
MD-04	24h change — stocks	Check priceChange24h on a stock asset	Calculated as ((currentPrice - previousClose) / previousClose) * 100 , rounded to 2 decimal places	
MD-05	updatedAt timestamp	Check any updated asset	updatedAt is current timestamp (within last minute)	
MD-06	Only mapped assets update	Add an asset with unmapped symbol to DB	That asset is NOT updated by refresh (no price data available)	
MD-07	Parallel fetching	Monitor refresh timing	CoinGecko and Yahoo fetches run in parallel (Promise.all), not sequentially	
MD-08	CoinGecko failure isolation	Block CoinGecko, refresh	Crypto prices stay stale, but stock/commodity prices still update — no crash	
MD-09	Yahoo failure isolation	Block Yahoo Finance, refresh	Stock prices stay stale, but crypto prices still update — no crash	
MD-10	Yahoo per-ticker isolation	One Yahoo ticker fails (e.g., UNG)	Other tickers (SPY, QQQ, etc.) still update — Promise.allSettled handles individual failures	
MD-11	10-second timeout	Simulate slow external API	Request aborts after 10 seconds, logs warning, continues to next source	

MD-12	Refresh return count	POST /api/markets/refresh	refreshed count = number of assets that had matching price data	
-------	----------------------	---------------------------	---	--

4.4 Evidence Signals

Endpoints: GET /api/signals/:assetId , GET /api/signals/feed/latest

ID	Test case	Steps	Expected result	P/F
SG-01	Get signals for asset	GET /api/signals/1	Returns {"signals": [...], "total": N} for asset ID 1	
SG-02	Signals limit param	GET /api/signals/1?limit=5	Returns at most 5 signals	
SG-03	Default limit is 20	GET /api/signals/1 (no limit)	Returns at most 20 signals	
SG-04	Signals sorted by date	Check signal order	Sorted by createdAt descending (newest first)	
SG-05	Latest feed	GET /api/signals/feed/latest	Returns up to 20 most recent signals across ALL assets	
SG-06	Empty signals	GET /api/signals/99999	Returns {"signals": [], "total": 0}	

4.5 Daily Briefing & Recommendations

Endpoints: GET /api/recommendations/briefing , POST /api/recommendations/scan , GET /api/recommendations/recommendations , GET /api/recommendations/events , GET /api/recommendations/watchlist , POST /api/recommendations/watchlist , DELETE /api/recommendations/watchlist/:id

ID	Test case	Steps	Expected result	P/F
BR-01	Briefing page loads — no data	GET /api/recommendations/briefing before any scan	Returns fallback: {"summary": "No briefing available yet...", "recommendations": [], "globalEvents": [], "tradeCount": 0, "watchCount": 0, "signalsProcessed": 0, "scanNumber": 0}	
BR-02	Trigger scan	POST /api/recommendations/scan	Returns {"status": "scan_started", "message": "Scanning global markets..."} immediately (scan runs in background)	

BR-03	Briefing after scan	GET /api/recommendations/briefing after scan completes (~60s)	Returns briefing with id , summary , tradeCount , watchCount , signalsProcessed , scanNumber , generatedAt , recommendations array, globalEvents array	
BR-04	Recommendation fields	Check any recommendation object	Contains: id , briefingId , type (trade/watch/avoid), urgency (high/medium/low), title , assetId , assetTitle , assetClass , sector , region , direction , aiProbability , marketPrice , edge , headline , why (array), historicalContext , bearCase , entryTrigger , confidence , window , urgencyReason , createdAt	
BR-05	List recommendations with filters	GET /api/recommendations/recommendations?type=trade&urgency=high	Returns only trade-type, high-urgency recommendations	
BR-06	List recommendations limit	GET /api/recommendations/recommendations?limit=5	Returns at most 5 recommendations, max cap 100	
BR-07	Recommendations sorted by confidence	GET /api/recommendations/recommendations	Sorted by confidence descending	
BR-08	Global events	GET /api/recommendations/events	Returns {"events": [...]} with id , title , region , impactLevel , detail , affectedAssets , direction , timeContext , scannedAt	
BR-09	Events limit	GET /api/recommendations/events?limit=5	Returns at most 5 events, max cap 50	
BR-10	Global event impact levels	Check event impactLevel field	Only: "critical" , "high" , "medium" , "low"	
BR-11	Global event direction	Check event direction field	Only: "bullish" , "bearish" , "mixed"	

BR-12	Get watchlist	GET /api/recommendations/watchlist	Returns {"watchlist": [...]} sorted by addedAt descending	
BR-13	Add to watchlist	POST /api/recommendations/watchlist with {"assetId": 1, "assetTitle": "Bitcoin", "assetClass": "crypto", "alertEdgeThreshold": 5.0, "notes": "test"}	Returns {"status": "added", "item": {...}}	
BR-14	Watchlist default threshold	POST /api/recommendations/watchlist with only {"assetId": 1}	Item created with alertEdgeThreshold: 5.0 default	
BR-15	Remove from watchlist	DELETE /api/recommendations/watchlist/{id}	Returns {"status": "removed"}	
BR-16	TRADE vs WATCH vs AVOID labels	Look at recommendation cards on frontend	Different visual treatment for type "trade" , "watch" , "avoid"	

4.6 Recommendations Scan Service (Internal Logic)

These tests validate the AI scan pipeline internals — not just the API response, but the quality and correctness of the data flowing through the system.

ID	Test case	Steps	Expected result	P/F
RS-01	Scan reads top 30 assets	Trigger scan, check console/DB	Scan queries up to 30 assets ordered by currentPrice descending	
RS-02	Scan reads latest 50 signals	Trigger scan, check console	Scan queries up to 50 signals ordered by createdAt descending	
RS-03	Global events scanned first	Trigger scan, watch console	Events scan runs before recommendation generation (events feed into recommendations)	
RS-04	Max 3 TRADE calls per briefing	Trigger multiple scans, check tradeCount	Never exceeds 3 TRADE type recommendations per briefing	
RS-05	Max 8 WATCH calls per briefing	Check recommendations per briefing	WATCH type count never exceeds 8	
RS-06	Scan number increments	Run 3 scans, check scanNumber each time	Each briefing gets incrementing scanNumber (1, 2, 3...)	
RS-07	Briefing summary generated	Check summary field after scan	3–4 sentence executive summary referencing specific assets and edge sizes	
RS-08	Empty summary fallback	Scan when AI returns no recommendations	Summary = "No significant opportunities identified in this scan. Markets appear fairly priced."	

RS-09	Single-rec summary fallback	Block summary AI call but recs succeed	Summary = "Today's scan identified N opportunities. Top call: [title] at [confidence]% confidence."	
RS-10	Events have required fields	Check events after scan in DB	Each event has title , region , impactLevel , detail , affectedAssets (array), direction , timeContext	
RS-11	Events region values	Check event region values	Only: "Middle East" , "Asia-Pacific" , "Europe" , "Americas" , "Africa" , "Global"	
RS-12	Recommendations linked to briefing	Check recommendations after scan	Every recommendation has briefingId matching the parent briefing id	
RS-13	JSON extraction handles markdown fences	Verify scan works when AI wraps response in ```json	Recommendations parse correctly (service strips markdown fences)	
RS-14	Scan failure doesn't crash server	Block Anthropic proxy, trigger scan	Console logs error, server continues running, next scan works when proxy restored	

4.7 Paper Trading & Portfolio

Endpoints: GET /api/portfolio , POST /api/portfolio/trade , POST /api/portfolio/close/:id , GET /api/portfolio/stats

ID	Test case	Steps	Expected result	P/F
PT-01	Portfolio initializes	GET /api/portfolio (first call)	Creates portfolio with balance: 10000 , initialBalance: 10000 , empty openTrades and closedTrades	
PT-02	Open long trade	POST /api/portfolio/trade with {"assetId": 1, "direction": "long", "amount": 200}	Returns {"trade": {...}, "balance": 9800, "message": "Opened long position on [name] for \$200.00"}	
PT-03	Open short trade	POST /api/portfolio/trade with {"assetId": 1, "direction": "short", "amount": 100}	Returns trade with direction: "short"	
PT-04	Trade direction validation	POST /api/portfolio/trade with {"direction": "invalid"}	Zod validation error (direction must be "long" or "short")	
PT-05	Insufficient balance	POST /api/portfolio/trade with amount > current balance	Returns {"error": "Insufficient balance"} with 400 status	
PT-06	Invalid asset	POST /api/portfolio/trade with {"assetId": 99999, ...}	Returns {"error": "Asset not found"} with 404 status	

PT-07	Portfolio shows open trade	GET /api/portfolio after opening trade	openTrades array contains the trade with correct assetId , assetName , assetSymbol , direction , entryPrice , quantity , status: "open"	
PT-08	Trade quantity calculation	Check trade quantity field	Equals amount / currentPrice (or amount if price is 0)	
PT-09	Close position	POST /api/portfolio/close/{trade-id}	Returns {"trade": {...}, "balance": N, "message": "Closed [direction] on [name]. PnL: \$X.XX"}	
PT-10	Closed trade has PnL	Check closed trade object	exitPrice , pnl , pnlPercent , closedAt all populated	
PT-11	Long PnL calculation	Close a long trade after price change	$PnL = (exitPrice - entryPrice) \times quantity$	
PT-12	Short PnL calculation	Close a short trade after price change	$PnL = -(exitPrice - entryPrice) \times quantity$	
PT-13	Close already-closed trade	POST /api/portfolio/close/{id} on closed trade	Returns {"error": "Trade already closed"} with 400 status	
PT-14	Close nonexistent trade	POST /api/portfolio/close/99999	Returns {"error": "Trade not found"} with 404 status	
PT-15	Balance updates after close	Check balance after closing trade	Balance = previous balance + (entryPrice $\times$ quantity + pnl)	
PT-16	Portfolio stats	GET /api/portfolio/stats	Returns {"totalTrades": N, "winRate": N, "avgReturn": N, "bestTrade": N, "worstTrade": N, "sharpeRatio": null, "balance": N, "totalPnL": N}	
PT-17	Win rate calculation	Close 5+ trades with mixed results	$winRate = (winners / closedTrades) \times 100$	
PT-18	Best/worst trade	Check stats after multiple trades	bestTrade = highest PnL, worstTrade = lowest PnL among closed trades	
PT-19	Trade fields complete	Check any trade object	Contains: id , assetId , assetName , assetSymbol , direction , entryPrice , exitPrice , quantity , pnl , pnlPercent , status , aiReasoning , openedAt , closedAt	
PT-20	Paper trade on mobile	Complete PT-02 through PT-09 on a smartphone	All steps work, UI is usable on small screen	

4.8 AI Coach

Endpoint: POST /api/coach/analyze

Body: {"assetId": number (optional), "question": string, "context": string (optional)}

Response: {"analysis": string, "recommendations": string[], "riskAssessment": string (optional), "confidence": number}

ID	Test case	Steps	Expected result	P/F
LC-01	Coach with asset context	POST /api/coach/analyze with {"assetId": 1, "question": "Should I buy this?"}	Returns {"analysis": "...", "recommendations": [...], "riskAssessment": "...", "confidence": N} within 30s	
LC-02	Coach without asset	POST /api/coach/analyze with {"question": "What markets look good right now?"}	Returns general market analysis (assetId is optional)	
LC-03	Coach with context	POST /api/coach/analyze with {"assetId": 1, "question": "Is this risky?", "context": "I'm a beginner"}	Analysis accounts for the provided context	
LC-04	Coach recommendations array	Check response recommendations field	Returns array of actionable string recommendations (max 5 items, parsed from bullet points in analysis)	
LC-05	Coach confidence score	Check response confidence field	Returns 0.75 on successful AI response	
LC-06	Coach error handling	POST /api/coach/analyze with empty body	Returns Zod validation error (question is required)	
LC-07	Coach uses Claude	Verify server console during request	Anthropic AI proxy called via Replit integrations with model claude-sonnet-4-20250514 , max_tokens: 1000	
LC-08	Coach on frontend	Navigate to /coach , ask a question	Response renders within 30s, specific to the question asked	
LC-09	Coach fallback on AI failure	Block Anthropic proxy, send coach request	Returns fallback: confidence: 0.3 , riskAssessment: "Unable to assess — AI service temporarily unavailable" , recommendations = 3 hardcoded tips, analysis starts with "I'm having trouble connecting to the AI service right now..."	
LC-10	Coach recommendation parsing	Ask question that produces numbered/bulleted list	Only lines starting with - , • , or 1. are parsed into recommendations , prefixes stripped	
LC-11	Coach system prompt role	Ask "what are you?" or "who are you?"	Response reflects trading coach persona: direct, data-driven, balanced (matches	

			COACH_PROMPT)	
LC-12	Context appended to prompt	Send with context: "I have \$5000 budget"	The context appears in the analysis (AI receives it as part of the user message)	

4.9 Live Trading & Risk Controls (E7)

Endpoints: GET /api/trading/accounts , GET /api/trading/route/:recommendationId , POST /api/trading/execute , GET /api/trading/pending , POST /api/trading/pending/:id/approve , POST /api/trading/pending/:id/reject , GET /api/trading/history , GET /api/trading/positions

ID	Test case	Steps	Expected result	P/F
KL-01	Accounts status	GET /api/trading/accounts	Returns {"accounts": {"usJurisdictionMode": bool, "primaryPlatform": "kalshi", "note": "...", "kalshi": {...}, "alpaca": {...}, "polymarket": {...}}}	
KL-02	Kalshi sub-object	Check accounts.kalshi	Contains status , legalStatus , depositMethod (when configured) or status , message , priority , legalStatus (when not)	
KL-03	Alpaca sub-object	Check accounts.alpaca	Contains status , legalStatus , assetTypes (when configured) or status , message , priority (when not)	
KL-04	Polymarket sub-object	Check accounts.polymarket	Contains status , legalStatus , restricted in US mode	
KL-05	Route a recommendation	GET /api/trading/route/{rec-id}	Returns {"recommendationId": N, "title": "...", "selectedPlatform": "...", "reason": "...", "tradeable": bool, "usJurisdictionMode": bool, "requireApproval": bool}	
KL-06	Route 404	GET /api/trading/route/99999	Returns {"error": "Recommendation not found"} with 404 status	
KL-07	Execute trade — risk blocked (low edge)	POST /api/trading/execute with rec that has edge < MIN_EDGE (default 5)	Returns {"success": false, "error": "Risk gate blocked: Edge X.X pts below minimum 5 pts"}	

KL-08	Execute trade — pending approval	POST /api/trading/execute with {"recommendationId": N, "amountUsd": 50, "overrideApproval": false} when REQUIRE_APPROVAL=true	Returns {"success": false, "error": "Order queued for your approval...", "status": "pending_approval"}	
KL-09	Execute trade — paper fallback	POST /api/trading/execute with {"recommendationId": N, "amountUsd": 50, "overrideApproval": true}	Returns {"success": true, "platform": "paper", "message": "Paper trade executed: \$50 on [title]", "reason": "..."}	
KL-10	Pending orders list	GET /api/trading/pending	Returns {"pending": [...]} with orders that have status: "pending_approval", sorted by createdAt descending	
KL-11	Approve pending order	POST /api/trading/pending/{id}/approve	Returns {"status": "approved", "orderId": N}, order status changes in DB	
KL-12	Approve logs live trade	After KL-11, GET /api/trading/history	Approved order appears as a live trade in history	
KL-13	Approve already-processed	POST /api/trading/pending/{id}/approve on already-approved order	Returns {"status": "approved", "orderId": N, "message": "Order already approved"}	
KL-14	Approve 404	POST /api/trading/pending/99999/approve	Returns {"error": "Pending order not found"} with 404 status	
KL-15	Reject pending order	POST /api/trading/pending/{id}/reject	Returns {"status": "rejected", "orderId": N}	
KL-16	Trade history	GET /api/trading/history	Returns {"trades": [...]} sorted by executedAt descending	
KL-17	History limit	GET /api/trading/history?limit=5	Returns at most 5 trades, max cap 100	
KL-18	History platform filter	GET /api/trading/history?platform=paper	Returns only paper-platform trades	
KL-19	Trade history fields	Check any trade in history	Contains: id , recommendationId , platform , assetId , assetTitle , direction , amountUsd , price , size , status , paperMode , aiProbability , aiEdge , confidence , orderId , ticker , executedAt	

KL-20	Open positions	GET /api/trading/positions	Returns {"positions": [...]} with trades that have status: "filled"	
KL-21	Risk gate — daily trade limit	Execute enough trades to exceed MAX_DAILY_TRADES (default 10)	Trade blocked: "Daily trade limit (10) reached"	
KL-22	\$10 live test trade (Core team)	Place smallest possible real trade through approval flow	Confirms platform integration end-to-end	

4.10 Risk Gate Deep Testing

These tests verify every risk check in the `checkRiskGate()` function. The risk config defaults are read from environment variables (or hardcoded defaults).

Risk Config Defaults: `MIN_EDGE=5` , `MIN_CONFIDENCE=65` , `MAX_POSITION_PCT=0.05` (5%), `MAX_DAILY_TRADES=10` , `DAILY_LOSS_LIMIT_PCT=0.10` (10%), `REQUIRE_APPROVAL=true` , `US_JURISDICTION_MODE=true`

ID	Test case	Steps	Expected result	P/F
RG-01	Minimum edge gate	Execute trade on rec with edge = 3 (below default 5)	Blocked: "Edge 3.0 pts below minimum 5 pts"	
RG-02	Edge gate passes	Execute trade on rec with edge = 8	Edge check passes (may still fail other checks)	
RG-03	Edge uses absolute value	Execute trade on rec with edge = -7		edge
RG-04	Minimum confidence gate	Execute trade on rec with confidence = 50 (below default 65)	Blocked: "Confidence 50% below minimum 65%"	
RG-05	Confidence gate passes	Execute trade on rec with confidence = 80	Confidence check passes	
RG-06	Max position size gate	Execute trade with <code>amountUsd = \$600</code> when portfolio = \$10,000	Blocked: "\$600 exceeds max position \$500 (5% of portfolio)" (default 5% × \$10K = \$500)	
RG-07	Position size passes	Execute trade with <code>amountUsd = \$400</code> when portfolio = \$10,000	Position check passes (\$400 < \$500 max)	
RG-08	Position check skipped if portfolio = 0	Execute when portfolio value = 0	Position check skipped (division by zero guard)	
RG-09	Daily loss limit gate	After losing \$1,100+ on a \$10,000 portfolio	Blocked: "Daily loss limit reached — trading paused" (default 10% = \$1,000 loss threshold)	
RG-10	Daily loss check skipped if portfolio = 0	Execute when portfolio value = 0	Loss limit check skipped	

RG-11	Daily trade count limit	Execute 11th trade in same calendar day	Blocked: "Daily trade limit (10) reached"	
RG-12	Daily count resets at midnight	Execute trades, wait past midnight UTC, execute again	Count resets — trades allowed again	
RG-13	All checks pass message	Execute when all checks pass	reason: "All risk checks passed"	
RG-14	Custom env overrides	Set <code>MIN_EDGE=10</code> in Secrets, restart server	Edge gate now requires 10 pts instead of 5	
RG-15	Approval gate toggle	Set <code>REQUIRE_APPROVAL=false</code> , execute trade	Trade executes immediately without pending approval step	
RG-16	Approval gate default	No <code>REQUIRE_APPROVAL</code> env var set	Defaults to <code>true</code> — all trades require approval	

4.11 Platform Router Deep Testing

These tests verify the keyword-based platform routing logic in `getBestPlatform()` .

ID	Test case	Steps	Expected result	P/F
PR-01	Stock/ETF Alpaca	Create rec with <code>assetClass: "stock"</code> or <code>sector: "equity"</code> , Alpaca configured	Routes to <code>"alpaca"</code> , reason mentions <code>"Stock/ETF market Alpaca"</code>	
PR-02	Stock paper if Alpaca missing	Same rec, Alpaca not configured	Routes to <code>"paper"</code> , reason: <code>"Alpaca not configured"</code>	
PR-03	Kalshi keyword match — fed	Create rec with title containing <code>"federal reserve rate cut"</code>	Routes to <code>"kalshi"</code> if configured	
PR-04	Kalshi keyword match — crypto	Create rec with title containing <code>"bitcoin"</code> or <code>"btc"</code>	Routes to <code>"kalshi"</code> if configured	
PR-05	Kalshi keyword match — economics	Create rec with title containing <code>"cpi"</code> , <code>"inflation"</code> , <code>"gdp"</code> , <code>"unemployment"</code>	Routes to <code>"kalshi"</code> if configured	
PR-06	Polymarket-only keywords	Create rec with title containing <code>"war"</code> , <code>"invasion"</code> , <code>"assassination"</code> , <code>"sports"</code> , <code>"oscar"</code>	Routes to <code>"polymarket"</code> if configured AND not US mode	
PR-07	US mode blocks Polymarket	Set <code>US_JURISDICTION_MODE=true</code> , rec matches Polymarket-only keywords	Routes to <code>"kalshi"</code> (not Polymarket), or <code>"paper"</code> if Kalshi not configured, reason mentions US jurisdiction	
PR-08	Non-US mode allows Polymarket	Set <code>US_JURISDICTION_MODE=false</code> , rec matches Polymarket-only keywords, Polymarket configured	Routes to <code>"polymarket"</code>	

PR-09	Kalshi fallback in non-US	Set <code>US_JURISDICTION_MODE=false</code> , Polymarket not configured	Falls back to "kalshi" if configured, reason: "Falling back to Kalshi"	
PR-10	No platform configured	Remove all platform credentials, route any rec	Routes to "paper" , tradeable: false	
PR-11	Kalshi overrides Polymarket for dual-match	Rec title matches both Kalshi keywords AND Polymarket-only keywords	Kalshi wins when <code>kalshiMatch && !polymarketOnlyMatch</code> is false — if only Polymarket matches and not Kalshi, Polymarket is used	

4.12 Market Radar (E8)

Endpoints: `GET /api/radar/alerts` , `GET /api/radar/prices` , `POST /api/radar/scan` , `GET /api/radar/chains/:assetId` , `GET /api/radar/chains` , `GET /api/radar/thresholds` , `GET /api/radar/history` , `GET /api/radar/status`

ID	Test case	Steps	Expected result	P/F
MR-01	Radar alerts — defaults	<code>GET /api/radar/alerts</code>	Returns <code>{"alerts": [...], "total": N, "generatedAt": "ISO timestamp"}</code> with default 4 hours lookback	
MR-02	Alerts — hours param	<code>GET /api/radar/alerts?hours=12</code>	Returns alerts from last 12 hours (clamped 1–24)	
MR-03	Alerts — type filter	<code>GET /api/radar/alerts?type=price_spike</code>	Only price spike alerts returned	
MR-04	Alerts — severity filter	<code>GET /api/radar/alerts?severity=high</code>	Only high severity alerts returned	
MR-05	Alerts — combined filters	<code>GET /api/radar/alerts?hours=8&type=volume_anomaly&severity=critical</code>	Filters applied together correctly	
MR-06	Alert fields complete	Check any alert object	Contains: <code>id</code> , <code>type</code> , <code>severity</code> , <code>assetId</code> , <code>assetLabel</code> , <code>title</code> , <code>pctChange</code> , <code>direction</code> , <code>priceStart</code> , <code>priceNow</code> , <code>windowMinutes</code> , <code>thresholdPct</code> , <code>volumeMultiplier</code> , <code>volumeType</code> , <code>confidence</code> , <code>reason</code> , <code>triggerAsset</code> , <code>triggerPct</code> , <code>chainAssets</code> ,	

			historicalNote , aiScanning , note , dataSource , createdAt	
MR-07	Prices with spike status	GET /api/radar/prices	Returns {"prices": [...], "total": N, "updatedAt": "ISO timestamp"}	
MR-08	Price fields complete	Check any price object	Contains: assetId , assetLabel , price , spikeDetected (boolean), pctChange , severity , threshold , updatedAt	
MR-09	Manual scan — start	POST /api/radar/scan	Returns {"status": "scan_started", "message": "Radar scan running in background..."}	
MR-10	Idempotent scan lock	POST /api/radar/scan twice rapidly	Second request returns {"status": "scan_already_running", "message": "A radar scan is already in progress..."}	
MR-11	Scan lock releases	Wait 30s after scan, POST /api/radar/scan again	New scan starts (lock released after previous completed)	
MR-12	Chain map — single asset	GET /api/radar/chains/brent_crude	Returns {"assetId": "brent_crude", "chains": [...], "total": 6, "note": "When BRENT CRUDE moves significantly..."} with 6 chain reactions	
MR-13	Chain map — unknown asset	GET /api/radar/chains/nonexistent	Returns {"assetId": "nonexistent", "chains": [], "total": 0, "note": "..."} }	

MR-14	Full chain map	GET /api/radar/chains	Returns {"chains": {...}, "totalAssets": 8, "note": "Full cross-asset correlation map..."} covering: brent_crude , wti_crude , ttf_gas , crypto_btc , stock_spy , wheat , gold , copper	
MR-15	Spike thresholds	GET /api/radar/thresholds	Returns {"thresholds": {...}, "total": 18, "note": "Configure spike sensitivity..." } with pct , window , severity per asset	
MR-16	History — defaults	GET /api/radar/history	Returns {"alerts": [...], "total": N, "byType": {...}, "bySeverity": {...}, "periodDays": 7} (default 7 days, 100 limit)	
MR-17	History — custom params	GET /api/radar/history?days=14&limit=50	Returns alerts from last 14 days, max 50 (clamped: days 1–30, limit 1–500)	
MR-18	History breakdowns	Check byType and bySeverity in history	Contains count maps like {"price_spike": 5, "volume_anomaly": 3}	
MR-19	Radar engine status	GET /api/radar/status	Returns {"engine": "E8 Market Radar", "scanFrequency": "Every 5 minutes", "assetsMonitored": 18, "chainMaps": 8, "sources": {...}, "activeSources": N, "totalSources": 5}	
MR-20	Status source details	Check individual source in status	Each source has status , tier , note fields	
MR-21	5-min auto-scan cron	Watch console for 5-min cron job	Radar scan fires every 5 minutes automatically	
MR-22	Prices sorted by	Check GET /api/radar/prices order	Sorted by `	pctChange

	pctChange			
MR-23	Threshold format string	Check price threshold field	Format: "X% / Ymin" (e.g., "2.0% / 15min") or "—" for assets without thresholds	

4.13 Radar Scan Engine Deep Testing

These tests verify the internal spike detection, chain reaction, volume anomaly, and alert storage logic.

ID	Test case	Steps	Expected result	P/F
RE-01	Price history accumulates	Trigger multiple radar scans over 10+ minutes	Price history builds up per asset (stored in memory, 2-hour window)	
RE-02	Price history pruned at 2 hours	Check after 2+ hours of scanning	Old price entries older than 2 hours are removed from memory	
RE-03	Spike detection — threshold match	Asset moves $\geq$ its threshold % within its window	price_spike alert generated with correct pctChange , direction , severity	
RE-04	Spike detection — below threshold	Asset moves $<$ its threshold %	No alert generated	
RE-05	Spike needs ≥ 2 price points	First scan for a brand-new asset	No spike detected (only 1 price point, no comparison possible)	
RE-06	Specific thresholds verified	GET /api/radar/thresholds , check brent_crude	pct: 2.0 , window: 15 , severity: "critical"	
RE-07	Specific thresholds verified	Check crypto_btc	pct: 3.0 , window: 15 , severity: "high"	
RE-08	Specific thresholds verified	Check stock_spy	pct: 1.0 , window: 10 , severity: "high"	
RE-09	Specific thresholds verified	Check crypto_sol	pct: 5.0 , window: 15 , severity: "medium"	
RE-10	Alert cooldown — 30 minutes	Trigger a spike alert, trigger another for same asset within 30 min	Second alert suppressed (cooldown active)	
RE-11	Alert cooldown expires	Wait 30+ minutes after an alert, trigger another spike	New alert generates (cooldown expired)	

RE-12	Chain reactions fire on spike	Spike detected on <code>brent_crude</code>	Chain reaction alerts generated for <code>wti_crude</code> , <code>stock_xle</code> , <code>airlines</code> , <code>tft_gas</code> , <code>nok_usd</code> , <code>cad_usd</code>	
RE-13	Chain confidence ≥ 60 filter	Check chain reactions for <code>brent_crude</code>	Only reactions with <code>confidence</code> ≥ 60 are included (all 6 of <code>brent_crude</code> 's chains qualify)	
RE-14	Chain confidence < 60 excluded	Check chain reactions for <code>gold_crypto_btc</code>	<code>crypto_btc</code> chain (confidence 52) is NOT fired as an alert	
RE-15	Chain reaction alert type	Check a chain reaction alert	<code>type</code> : "chain_reaction" , <code>severity</code> : "medium" , has <code>triggerAsset</code> , <code>triggerPct</code> , <code>confidence</code> , <code>reason</code>	
RE-16	Volume anomaly detection	Scan when <code>SPY/QQQ/GLD/USO/XLE</code> volume $>$ threshold $\times$ 30-day average	<code>volume_anomaly</code> alert generated with <code>volumeMultiplier</code> , <code>volumeType</code> : "equity_volume"	
RE-17	Volume severity escalation	Volume $\geq$ threshold $\times 1.5$	Severity escalates from "medium" to "high"	
RE-18	Volume cooldown	Volume anomaly fires, trigger another scan immediately	Second volume alert suppressed (same cooldown system, keyed <code>vol_[assetId]</code>)	
RE-19	Volume thresholds per asset	Check volume detection	<code>SPY/QQQ</code> : 2.5 $\times$, <code>XLE</code> : 3.0 $\times$, <code>BTC/ETH</code> : 2.0 $\times$, <code>wheat/gold/WTI</code> : per config, default fallback: 3.0 $\times$	
RE-20	Volume needs 10+ days data	Asset with < 10 days of volume history	No volume anomaly generated (insufficient data)	
RE-21	Historical note attached	Spike on <code>brent_crude</code>	Alert <code>historicalNote</code> = "Moves of this magnitude without news have preceded breaking geopolitical events within 60 min..."	
RE-22	Historical note — BTC	Spike on <code>crypto_btc</code>	Alert <code>historicalNote</code> references CME futures premium and exchange reserves	
RE-23	Historical note — no data	Spike on asset without historical pattern	<code>historicalNote</code> = "Monitoring for catalyst — no historical pattern data available for this asset yet."	
RE-24	Alert ID is unique	Check alert IDs	MD5 hash of <code>assetId</code> + ISO timestamp , first 10 chars	
RE-25	Alert deduplication in DB	Same alert ID inserted twice	<code>onConflictDoUpdate</code> updates <code>title</code> and <code>severity</code> only — no duplicate rows	
RE-26	Asset label formatting	Check alert <code>assetLabel</code> for <code>crypto_btc</code> and <code>stock_spy</code>	Underscores replaced with spaces, <code>crypto_</code> and <code>stock_</code> prefixes stripped, uppercased: "BTC" , "SPY"	

RE-27	Spike title format	Check price spike alert title	Format: "[LABEL] [up/down] [X.X]% in [N] minutes"	
RE-28	Chain assets in spike alert	Check spike alert chainAssets field	Lists up to 6 chain reaction asset IDs from CHAIN_REACTIONS map	
RE-29	AI scanning note	Check spike alert aiScanning	Format: "Scanning Reuters, AP, Bloomberg for [LABEL] catalyst..."	
RE-30	Data source attribution	Check spike alert dataSource	"Yahoo Finance / CoinGecko"	
RE-31	Zero/negative price skipped	Asset returns price = 0 or negative from API	Skipped entirely — no price history update, no spike check	

4.14 Scheduler & Background Jobs

ID	Test case	Steps	Expected result	P/F
SC-01	Market refresh fires every 5 min	Watch server console for 10+ minutes	At least 2 market refresh cycles complete (logged as "Market data refresh complete")	
SC-02	Recommendation scan fires every 30 min	Watch console for 35+ minutes	Recommendation scan fires once (logged as "E6: Briefing generated")	
SC-03	Radar scan fires every 5 min	Watch console for 10+ minutes	At least 2 radar scans complete (logged as "E8: Radar scan complete" or "E8: No anomalies detected this scan")	
SC-04	Initial refresh at 3 seconds	Restart app, watch console with timestamp	"Running initial market data refresh..." appears ~3 seconds after "Scheduler started"	
SC-05	Market refresh lock	Trigger manual POST /api/markets/refresh while auto-refresh running	If isRefreshing is true, the scheduled refresh skips: "Market refresh already in progress, skipping"	
SC-06	Recommendation scan lock	Trigger POST /api/recommendations/scan while auto-scan running	If isScanning is true, skips: "Recommendations scan already in progress, skipping"	
SC-07	Radar scan lock	Trigger POST /api/radar/scan while auto-scan running	If isRadarScanning is true, skips: "Radar scan already in progress, skipping"	
SC-08	Lock releases on success	Wait for a scheduled job to complete	Lock flag resets to false , next scheduled run proceeds normally	

SC-09	Lock releases on failure	Force a job to fail (e.g., kill DB connection briefly)	Lock flag still resets to <code>false</code> (try/finally), next run proceeds — NOT permanently locked	
SC-10	Market refresh error logging	Force refresh failure	Console shows <code>"Scheduled market refresh failed"</code> with error message, app continues	
SC-11	Recommendation scan error logging	Force scan failure	Console shows <code>"Scheduled recommendations scan failed"</code> with error message, app continues	
SC-12	Radar scan error logging	Force radar failure	Console shows <code>"Scheduled radar scan failed"</code> with error message, app continues	
SC-13	Radar scan logs alert count	Trigger radar scan when spikes are detected	Console shows <code>"Radar scan complete"</code> with <code>{count: N}</code>	
SC-14	All 3 jobs run independently	Observe console over 30+ minutes	Market refresh and radar scan fire every 5 min (independently), recommendation scan fires every 30 min — no interference between jobs	
SC-15	Server stays stable over time	Leave app running for 2+ hours	No memory leaks, no crashes, all scheduled jobs continue firing	

4.15 API Schema & Error Handling

ID	Test case	Steps	Expected result	P/F
API-01	All GET endpoints return valid JSON	Hit every GET endpoint listed above	Valid JSON response, correct HTTP status codes	
API-02	POST endpoints validate input (Zod)	Send <code>POST /api/portfolio/trade</code> with <code>{"direction": "invalid"}</code>	Returns Zod validation error, not 500	
API-03	POST with empty body	Send <code>POST /api/coach/analyze</code> with <code>{}</code>	Returns Zod validation error (question is required string)	
API-04	404 on unknown routes	<code>GET /api/nonexistent</code>	Returns 404, not 500 or HTML	
API-05	500 errors include message	Trigger a server error	Returns <code>{"error": "descriptive message"}</code>	
API-06	Numeric ID coercion	<code>GET /api/markets/abc</code>	Returns Zod coercion error or 500 (not crash)	
API-07	Response shapes match Zod schemas	Compare actual response to generated Zod schemas in <code>lib/api-zod</code>	All fields present and correctly typed	

4.16 Frontend Page & Route Testing

All pages use Wouter for routing with a shared `Layout` component. React Query caches data for 5 minutes (`staleTime: 5min` , `refetchOnWindowFocus: false`). Every page must load without blank screens, console errors, or uncaught exceptions on all target browsers and devices.

Routes: `/` (Scanner), `/market/:id` (Market Detail), `/portfolio` (Portfolio), `/coach` (Coach), `/briefing` (Briefing), `/radar` (Radar), catch-all (Not Found)

4.16.1 Page Load & Render

ID	Test case	Steps	Expected result	P/F
FE-01	Scanner page loads	Navigate to <code>/</code>	Markets list renders with asset cards/rows, sector filter visible, sort dropdown visible, no console errors	
FE-02	Market Detail page loads	Navigate to <code>/market/1</code>	Asset name, price, AI probability, edge, signals list, related markets all render	
FE-03	Portfolio page loads	Navigate to <code>/portfolio</code>	Balance displayed (\$10,000 initial), open trades section visible, closed trades section visible, stats section visible	
FE-04	Coach page loads	Navigate to <code>/coach</code>	Question input field visible, submit button visible, no pre-existing errors	
FE-05	Briefing page loads	Navigate to <code>/briefing</code>	Briefing summary shown (or "No briefing available yet" message), recommendations section visible, events section visible	
FE-06	Radar page loads	Navigate to <code>/radar</code>	Alerts list visible, price monitor visible, chain reaction map accessible, no console errors	
FE-07	Not Found page	Navigate to <code>/nonexistent-route</code>	Renders 404 / Not Found page, does NOT show blank screen or crash	
FE-08	Market Detail — invalid ID	Navigate to <code>/market/99999</code>	Shows error state (e.g., "Market not found"), does NOT crash or show blank page	

4.16.2 Navigation & Routing

ID	Test case	Steps	Expected result	P/F
FE-09	Nav links work	Click each navigation link in Layout header/sidebar	Correct page renders, URL updates, no full-page reload (client-side routing)	
FE-10	Scanner Market Detail	Click any market row on Scanner page	Navigates to <code>/market/{id}</code> with correct asset details	
FE-11	Market Detail back	Click back button or browser back	Returns to Scanner with previous scroll position and filters preserved	
FE-12	Briefing Market Detail	Click a recommendation that links to an asset	Navigates to correct market detail page	

FE-13	Direct URL access	Type <code>/portfolio</code> directly in browser address bar	Portfolio page loads correctly (not just via nav clicks)	
FE-14	Deep link — market detail	Share <code>/market/3</code> URL with another tester	Page loads correctly with asset ID 3 data	
FE-15	BASE_URL handled	Check that all routes respect <code>import.meta.env.BASE_URL</code>	Routes work through Replit's proxy path prefix, not just at root <code>/</code>	

4.16.3 Interactive Elements & State

ID	Test case	Steps	Expected result	P/F
FE-16	Scanner — sector filter	Select different sector from dropdown/tabs	Market list updates to show only that sector, no full page reload	
FE-17	Scanner — sort change	Change sort option (alpha score, price change, name)	Market list re-orders immediately	
FE-18	Market Detail — Score AI button	Click "Score" / "Re-score" button on market detail	Loading indicator appears, scoring completes, probability/edge/signals update in place	
FE-19	Portfolio — open trade modal	Click trade button on any asset	Trade modal/form appears with asset name, direction selector (long/short), amount input	
FE-20	Portfolio — submit trade	Fill in trade form, submit	Trade appears in open trades list, balance decreases, toast/confirmation shown	
FE-21	Portfolio — close trade	Click close button on an open trade	Trade moves to closed trades, PnL displayed, balance updates	
FE-22	Coach — submit question	Type a question, click submit	Loading indicator appears, AI response renders within 30s with analysis + recommendations	
FE-23	Coach — empty submit	Click submit with empty question field	Validation message shown, request not sent	
FE-24	Briefing — trigger scan	Click "Scan" button on briefing page	Loading indicator, scan starts in background, new recommendations appear after completion	
FE-25	Briefing — watchlist add	Click "Watch" or add-to-watchlist on a recommendation	Item added, confirmation shown, watchlist updates	
FE-26	Radar — manual scan	Click scan/refresh button on radar page	Scan triggers, loading indicator, new alerts appear after completion	
FE-27	Radar — filter alerts	Filter by type (price spike, volume anomaly, chain reaction) or severity	Alert list updates to show only matching alerts	

FE-28	Radar — chain map view	Click on a chain reaction asset or navigate to chain map	Chain reaction map renders with linked assets, directions, confidence	
FE-29	Toast notifications	Trigger actions that show toasts (trade, scan, error)	Toasts appear, are readable, auto-dismiss after a few seconds	
FE-30	Tooltip component	Hover over elements with tooltips	Tooltips render correctly, positioned properly, don't clip off screen	

4.16.4 Loading, Error & Empty States

ID	Test case	Steps	Expected result	P/F
FE-31	Scanner — loading state	Hard refresh / page	Loading spinner or skeleton shown while markets fetch	
FE-32	Market Detail — loading state	Navigate to /market/1	Loading indicator while data fetches	
FE-33	Coach — loading state	Submit a question	Loading indicator while AI responds (up to 30s)	
FE-34	Briefing — empty state	View briefing before any scan has run	Friendly message like “No briefing available yet”, not a crash or blank	
FE-35	Portfolio — empty state	View portfolio with no trades	Shows \$10,000 balance, empty trade lists with helpful message	
FE-36	Radar — no alerts state	View radar when no alerts exist	Friendly “No alerts detected” message, not a blank page	
FE-37	API error state	Kill the API server, navigate to any page	Error message rendered on page (e.g., “Failed to load”), no uncaught exception in console	
FE-38	Network timeout	Throttle to very slow connection, load scanner	Page eventually loads or shows timeout error — does NOT hang forever	

4.16.5 React Query Caching Behavior

ID	Test case	Steps	Expected result	P/F
FE-39	5-minute stale time	Load scanner, wait 3 minutes, navigate away and back	Markets load instantly from cache (no new API call)	
FE-40	Cache expires after 5 min	Load scanner, wait 6 minutes, navigate away and back	Fresh API call fires, data updates	
FE-41	No refetch on window focus	Load scanner, switch to another tab, switch back	No new API call triggered on tab focus (refetchOnWindowFocus is disabled)	
FE-42	Manual refresh updates cache	Click refresh/re-fetch button (if available on any page)	New data fetched and cache updated, UI reflects changes	

4.16.6 Responsive Design & Cross-Browser

ID	Test case	Steps	Expected result	P/F
FE-43	Desktop layout	View every page on 1280px+ screen	Full layout renders, sidebar/header visible, tables/cards properly spaced	
FE-44	Tablet layout	View every page on 768px–1024px screen	Layout adapts, no horizontal scroll, all content accessible	
FE-45	Mobile layout	View every page on 375px–414px screen	Layout stacks vertically, buttons tappable (min 44px), text readable without zoom	
FE-46	Chrome desktop	Full test pass on Chrome 120+	All pages load, all interactions work	
FE-47	Firefox desktop	Full test pass on Firefox latest	All pages load, all interactions work	
FE-48	Safari desktop	Full test pass on Safari latest	All pages load, all interactions work	
FE-49	Chrome mobile (Android)	Full test pass on Android Chrome	All pages load, touch interactions work, modals usable	
FE-50	Safari mobile (iOS)	Full test pass on iOS Safari	All pages load, touch interactions work, no viewport issues	
FE-51	Edge desktop	Quick smoke test on Edge	All pages load, no major visual issues	
FE-52	Slow 3G connection	Throttle to 3G, navigate through all pages	Pages eventually load with loading states, no timeouts under 30s	

5. Phase 3 — User Acceptance Testing (Week 4)

Finance students test real usage scenarios without being told what to do. The goal is discovering where real users get confused, what they misunderstand, and what delights them. Observers take notes — do NOT help unless the tester is completely stuck.

5.1 Session Setup

- **Duration:** 90 minutes per session
- **Group size:** 3–5 students per session (so you can observe body language)
- **Device:** student's own phone or a provided tablet — not a desktop (most real users will be mobile)
- **Recording:** screen recording on with permission, or observer takes timestamped notes
- **Starting state:** fresh \$10,000 paper balance, no prior explanation of features

5.2 UAT Scenario Scripts

Scenario A — Beginner: First Trade (30 min)

Read this to the tester:

You've heard that AI can help you find trading opportunities. You've just opened Alpha Lens for the first time. Explore the app and try to make your first simulated trade. Think out loud as you go — say what you're looking at, what confuses you, what makes sense.

Observer watches for:

- Does the tester find the Scanner without help?
- Do they understand what AI probability vs Market price means?
- Do they find the Trade button without prompting?
- Do they understand the trade modal? Do they know what “long” vs “short” means?
- Do they know no real money is involved?
- Does the Portfolio page make sense after their first trade?

Scenario B — Intermediate: Acting on a Recommendation (30 min)

Read this to the tester:

The AI has just flagged a trading opportunity in the Daily Briefing. Your job is to evaluate whether you agree with the AI's reasoning and decide whether to paper trade it. Use whatever information you can find in the app to make your decision.

Observer watches for:

- Does the tester navigate to the briefing page without help?
- Do they read the historical context and bear case?
- Do they click through to the asset detail page for more information?
- Do they use the AI Coach before deciding?
- What is their decision rationale? Does the edge concept make sense to them?

Scenario C — Advanced: Market Radar & Platform Review (30 min)

Read this to the tester:

You are a finance professional reviewing this AI trading tool. Spend 15 minutes exploring the Market Radar feature — what alerts does it surface, and do the chain reaction maps make sense? Then spend 15 minutes reviewing the live trading pipeline: how does the approval flow work, and would you trust it with real capital?

Observer watches for:

- Do they understand the radar alert types (price spike, volume anomaly, chain reaction)?
- Do the chain reaction maps make intuitive sense to them?
- What do they question about the AI methodology?
- Do they trust the evidence sources shown?
- What would make them trust it more?
- What features do they wish existed?

5.3 Post-Session Survey

Every tester completes this after their session. Score 1–5 (1=strongly disagree, 5=strongly agree):

Question	Score 1–5	Comments
I understood what Alpha Lens does within the first 2 minutes		
I understood the difference between AI probability and market price		

I understood that paper trading uses no real money		
I understood what “long” and “short” mean in this context		
The Daily Briefing recommendations were clear and actionable		
The AI Coach explanation helped me understand the trade		
The Market Radar alerts were useful and understandable		
I could complete a paper trade without help		
I would use this app for real investment research		
The app worked well on my device		
I would recommend this to other finance students		
I would trust this AI with real trading decisions after more practice		

Open questions (written response):

- What was the most confusing part of the app?
- What was the most impressive feature?
- What feature is missing that you would want?
- Would you use this on your phone? Why or why not?

6. Phase 4 — Pre-Launch Validation (Week 5)

6.1 Load Testing

Test	Method	Target	Pass criterion
10 simultaneous users	Open app on 10 devices at same time	All users browsing scanner	No timeouts, response < 3s
5 concurrent AI scores	5 testers click Re-score AI simultaneously	All score different markets	All complete within 120s, no crashes
Scheduler under load	Background jobs fire while 10 users active	All 3 cron jobs run concurrently with user traffic	Jobs complete, no user-facing slowdown
Concurrent market refreshes	Trigger <code>POST /api/markets/refresh</code> while cron is refreshing	<code>isRefreshing</code> lock	Only one refresh runs, cron skips with log message
Concurrent recommendation scans	Trigger <code>POST /api/recommendations/scan</code> while cron is scanning	<code>isScanning</code> lock	Only one scan runs, cron skips with log message

Concurrent radar scans	3 users trigger <code>POST /api/radar/scan</code> at same time	<code>isRadarScanning</code> lock	Only one scan runs, others return <code>scan_already_running</code>
Sustained 2-hour run	Leave app running with all cron jobs active, 5 users browsing	Stability over time	No memory leaks, no crashes, all jobs continue firing on schedule

6.2 Security Checklist

Check	How to verify	Pass criterion
No API keys in frontend code	View page source in browser, search for key names	Zero API keys visible in browser
Replit Secrets not in code	Search codebase for hardcoded credentials	No real credentials in committed files
Anthropic proxy secured	Verify AI calls go through Replit proxy, not direct	No Anthropic API key exposed in client code
CORS configuration	Check <code>artifacts/api-server/src/app.ts</code> CORS config	Tighten to specific domain before launch
Paper trade can't exceed balance	<code>POST /api/portfolio/trade</code> with <code>amount > balance</code>	Returns <code>{"error": "Insufficient balance"}</code> with 400
Live trade approval gate works	<code>POST /api/trading/execute</code> with <code>overrideApproval: false</code>	Order queued as <code>pending_approval</code> , not fired immediately
Risk controls enforced server-side	Call <code>POST /api/trading/execute</code> directly via curl bypassing frontend	Backend rejects based on risk gate regardless of frontend state
All 5 risk checks enforced	Test each: min edge, min confidence, max position %, daily loss limit, daily trade count	Each independently blocks a trade when threshold exceeded
Radar scan lock prevents abuse	Rapid-fire <code>POST /api/radar/scan</code>	Lock prevents concurrent scans, returns <code>scan_already_running</code>
Scheduler locks prevent abuse	Rapid-fire all 3 scan/refresh endpoints	All 3 locks work independently, prevent concurrent executions
Zod validation on all POST routes	Send invalid types to every POST endpoint	Server-side Zod schemas reject invalid data, return descriptive errors
Risk config env vars secured	Check risk defaults via behavior testing	<code>MIN_EDGE</code> , <code>MIN_CONFIDENCE</code> , <code>MAX_POSITION_PCT</code> , <code>MAX_DAILY_TRADES</code> , <code>DAILY_LOSS_LIMIT_PCT</code> not exposed in any API response

Platform credentials not exposed	GET /api/trading/accounts	Response shows status (configured/not_configured) but never returns actual API keys or passwords
----------------------------------	---------------------------	--

6.3 Data Quality Validation

Check	Criteria	Tester
AI probabilities are calibrated	Sample 10 markets: AI probability within 15pts of market on well-traded markets	Core + PH Data
Evidence sources are real	Check signal source values — should reference real data sources	All teams
AI generates 3–6 signals per scoring	Score 5 different markets, check signal count	Each scoring creates 3–6 new signals with distinct types
Recommendations have historical context	Every TRADE type recommendation should include historicalContext field with year citations	Core
Max 3 TRADE calls per briefing	Run 5+ scans, check each briefing tradeCount	Never exceeds 3
Bear case present on trade calls	Every type: "trade" recommendation should have bearCase populated	Finance students
Coach commentary is specific	Coach analysis must reference the actual market/question, not generic advice	All teams
Coach fallback works gracefully	Temporarily block AI, send coach request	Returns helpful fallback with 3 tips and low confidence (0.3)
Radar alerts are accurate	Cross-check 5+ price spike alerts against actual market data	PH Data
Chain reaction maps are logical	Review all 8 chain maps via GET /api/radar/chains	Correlations make economic sense (oil airlines bear, BTC ETH bull, etc.)
Volume anomaly thresholds reasonable	Review volume multipliers for SPY/QQQ/GLD/USO/XLE	Multipliers between 2.0× and 3.0× — not too sensitive, not too loose
Historical patterns accurate	Check 5 historical notes (brent_crude, crypto_btc, wheat, gold, stock_spy)	Each note references real historical events with accurate context
Alert cooldown prevents spam	Trigger multiple scans in 30 min with same spikes	Same asset doesn't generate duplicate alerts within 30-minute window

6.4 Database Schema Validation

Check	How to verify	Pass criterion
All Drizzle tables created	<code>pnpm --filter @workspace/db run push</code>	No migration errors
assets table populated	Seed script + <code>GET /api/markets</code>	12+ seeded assets present
signals table populated	<code>GET /api/signals/feed/latest</code>	11+ seeded signals present
trades table functional	Open and close a paper trade	Trade records created with correct entry/exit prices, PnL calculated
portfolio balance tracking	<code>GET /api/portfolio</code> after trades	Balance updates accurately
daily_briefings stored	<code>POST /api/recommendations/scan</code> , then <code>GET /api/recommendations/briefing</code>	Briefing record with summary, scanNumber, generatedAt
recommendations linked	Check recommendations after scan	Each has <code>briefingId</code> linking to parent briefing
global_events stored	<code>GET /api/recommendations/events</code> after scan	Events with impact levels and affected assets
watchlist CRUD	Add then remove items	<code>POST</code> returns item, <code>DELETE</code> returns <code>{"status": "removed"}</code>
live_trades tracked	<code>POST /api/trading/execute</code> , then <code>GET /api/trading/history</code>	Trade recorded with platform, amount, AI metadata, <code>paperMode</code> flag
pending_orders flow	Execute with <code>overrideApproval: false</code> , approve, then reject another	Status transitions: <code>pending_approval</code> <code>approved</code> / <code>rejected</code>
radar_alerts stored	<code>POST /api/radar/scan</code> , then <code>GET /api/radar/alerts</code>	Alerts with severity, asset data, timestamps
radar_alerts dedup	Insert alert with same ID twice	<code>onConflictDoUpdate</code> updates <code>title</code> and <code>severity</code> only — no duplicate rows
scan number incrementing	Run 3 scans, check daily_briefings table	<code>scanNumber</code> increments: 1, 2, 3
daily trade count query	<code>getDailyTradeCount()</code> after trades	Returns count of live trades with <code>executedAt</code> <code>>=</code> midnight today

6.5 Launch Sign-Off Checklist

Item	Signed off by	Date	Status
Phase 1 all checks green	PH Lead QA		

Phase 2 zero critical bugs open	PH Lead QA		
Phase 3 average UAT score >= 3.5/5	Core Team		
Load test 10 users — no crashes	PH Lead QA		
Security checklist complete	Core Team		
\$10 live Kalshi test trade successful	Core Team		
AI recommendation quality approved	Core Team		
All 5 risk checks verified server-side	PH API Tester		
Platform router tested for all keyword categories	PH API Tester		
Market Radar alerts accurate and timely	PH Data + Core		
Chain reaction maps reviewed and validated	Core Team		
Scheduler runs stable for 2+ hours	PH Lead QA		
Alert cooldown system working (30-min dedup)	PH API Tester		
AI fallback behavior verified (scoring + coach)	PH API Tester		
Financial disclaimer visible on all pages	All teams		
Replit Deployment always-on confirmed	Core Team		
TypeScript typecheck passes with zero errors	PH Lead QA		
Drizzle schema push completes cleanly	PH Lead QA		
Backup/restore procedure documented	PH Lead QA		
Support channel established for testers	Core Team		

7. Bug Reporting & Communication

7.1 Bug Severity Levels

Severity	Definition	Examples	Response time
P1 Critical	App unusable, data loss, security issue, money at risk	App crashes, live trade fires without approval, credentials exposed, radar scan crashes server, scheduler lock permanently stuck	Fix before any launch
P2 High	Major feature broken, blocks testing path	Scanner blank, paper trade fails, briefing doesn't load, radar alerts missing, market refresh stops working	Fix within 24 hours
P3 Medium	Feature works but incorrectly or	Edge calculation off, wrong color badge, coach note generic, PnL math wrong, chain reaction	Fix within 3 days

	inconsistently	map incomplete, volume threshold too sensitive	
P4 Low	Minor UI issue, cosmetic, not blocking	Typo, alignment off on one browser, color slightly wrong, alert label capitalization	Fix before launch
Enhancement	Not a bug — a missing feature or improvement	User wants X feature, wants Y to work differently	Log for roadmap

7.2 Bug Report Template

Every bug report must include all of the following:

ID: [auto-number]
Severity: P1 / P2 / P3 / P4
Title: Short description (e.g. 'Paper trade modal doesn't close on mobile')
Reporter: Name + team
Date/Time: When observed
Device + Browser: e.g. iPhone 14, Safari 17
Steps to reproduce: Numbered steps that reproduce the bug every time
Expected result: What should happen
Actual result: What actually happened
Screenshot/Recording: Attached
Notes: Any additional context

7.3 Communication Channels

Channel	Purpose	Frequency
Shared bug tracking sheet (Google Sheets)	All bugs logged here — one row per bug, status tracked	Updated in real time
Group chat (WhatsApp / Slack)	Quick questions, screenshots, daily standup notes	Daily during testing weeks
Weekly video call	Phase review, bug triage, decisions on P3/P4 fixes	Once per week
Philippines team standup	Daily 15-min sync during Phase 1 and 2	Daily 9am Manila time
Finance student debrief	Group discussion after each UAT session	After each session

7.4 Time Zone Coordination

Location	Time Zone	Overlap with US Eastern
Manila, Philippines	PHT (UTC+8)	Night in US = Day in Philippines. US 9pm ET = Manila 9am next day
US Eastern	ET (UTC-4 summer / UTC-5 winter)	Core working hours for US team

Suggested overlap window	US 8–10am ET = Manila 8–10pm PHT	2-hour daily sync window — adjust as needed
--------------------------	----------------------------------	---

Appendix A — Complete API Endpoint Reference

All endpoints are prefixed with `/api` (e.g., `GET /api/markets`).

Markets

Method	Path	Description
GET	<code>/markets</code>	List all markets (query: <code>sector</code> , <code>sort</code> , <code>limit</code>)
GET	<code>/markets/:id</code>	Market detail with signals and related markets
POST	<code>/markets/:id/score</code>	Trigger AI scoring via Claude
POST	<code>/markets/refresh</code>	Refresh all market data from external sources

Signals

Method	Path	Description
GET	<code>/signals/feed/latest</code>	Latest 20 signals across all assets
GET	<code>/signals/:assetId</code>	Signals for a specific asset (query: <code>limit</code>)

Portfolio

Method	Path	Description
GET	<code>/portfolio</code>	Portfolio with open/closed trades and balance
POST	<code>/portfolio/trade</code>	Open a paper trade (body: <code>assetId</code> , <code>direction</code> , <code>amount</code>)
POST	<code>/portfolio/close/:id</code>	Close a paper trade
GET	<code>/portfolio/stats</code>	Performance statistics

Coach

Method	Path	Description
POST	<code>/coach/analyze</code>	AI coaching analysis (body: <code>question</code> , optional <code>assetId</code> , <code>context</code>)

Recommendations

Method	Path	Description
GET	<code>/recommendations/briefing</code>	Latest AI intelligence briefing with recommendations and events

POST	/recommendations/scan	Trigger a new AI recommendations scan (runs in background)
GET	/recommendations/recommendations	List recommendations (query: type , urgency , limit)
GET	/recommendations/events	Recent global market events (query: limit)
GET	/recommendations/watchlist	Get watchlist items
POST	/recommendations/watchlist	Add to watchlist (body: assetId , optional assetTitle , assetClass , alertEdgeThreshold , notes)
DELETE	/recommendations/watchlist/:id	Remove from watchlist

Trading

Method	Path	Description
GET	/trading/accounts	Platform status (Kalshi/Alpaca/Polymarket) with US jurisdiction info
GET	/trading/route/:recommendationId	Preview platform routing decision
POST	/trading/execute	Execute a live trade (body: recommendationId , amountUsd , optional platform , overrideApproval)
GET	/trading/pending	Get pending approval orders
POST	/trading/pending/:id/approve	Approve pending order
POST	/trading/pending/:id/reject	Reject pending order
GET	/trading/history	Live trade history (query: limit , platform)
GET	/trading/positions	Open live trading positions (status: "filled")

Radar

Method	Path	Description
GET	/radar/alerts	Recent radar alerts (query: hours , type , severity)
GET	/radar/prices	Current prices with spike status for all monitored assets
POST	/radar/scan	Manually trigger radar scan (idempotent lock)
GET	/radar/chains/:assetId	Chain reaction map for a specific asset
GET	/radar/chains	Full cross-asset chain reaction map
GET	/radar/thresholds	Spike detection thresholds per asset
GET	/radar/history	Historical alerts for trend analysis (query: days , limit)
GET	/radar/status	Radar engine status and data source availability

Health

Method	Path	Description
GET	/healthz	Health check

Appendix B — Service Architecture Reference

Service	File	Cron Schedule	Lock Flag	Key Logic
Market Data	market-data.ts	Every 5 min	isRefreshing	Fetches CoinGecko (BTC/ETH/SOL) + Yahoo Finance (SPY/QQQ/GLD/USO/UNG/EURUSD) in parallel, updates <code>assetsTable</code>
Recommendations	recommendations.ts	Every 30 min	isScanning	Reads top 30 assets + 50 signals, calls Claude for events + recs + summary, stores to DB
Market Radar	market-radar.ts	Every 5 min	isRadarScanning	Fetches all prices (crypto + 17 Yahoo tickers), checks 18 spike thresholds, fires chain reactions (confidence ≥ 60), detects volume anomalies (5 tickers), 30-min alert cooldown, <code>onConflictDoUpdate</code> dedup
AI Scoring	scoring.ts	On-demand	—	Calls Claude for probability + signals, updates market fields, generates 3–6 evidence signals, fallback scoring on AI failure
AI Coach	coach.ts	On-demand	—	Calls Claude with trading mentor prompt, parses bullet points for recommendations (max 5), confidence = 0.75, fallback with 3 hardcoded tips on failure
Scheduler	scheduler.ts	Startup	—	Orchestrates all cron jobs, fires initial market refresh 3s after boot via <code>setTimeout</code>

Final Note — Paper Trading Explanation for All Testers

Every tester must understand this before starting: Paper trading in Alpha Lens is a fully digital, in-app simulation. No broker is called. No real money changes hands. The \$10,000 balance is fictional and exists only within the app. A paper trade is executed by clicking a button on your phone or computer screen — exactly like a real trade will be, but with zero financial risk.

